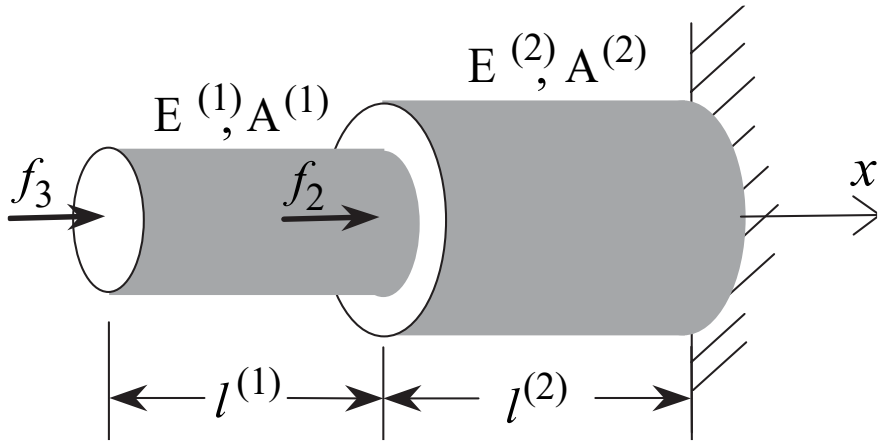
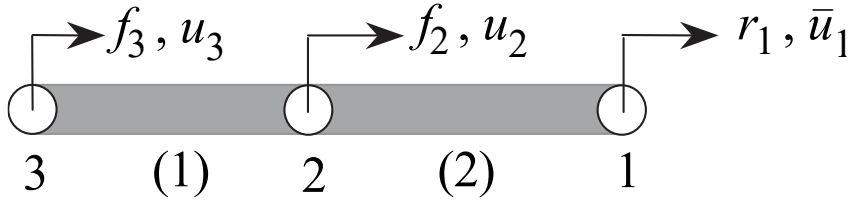
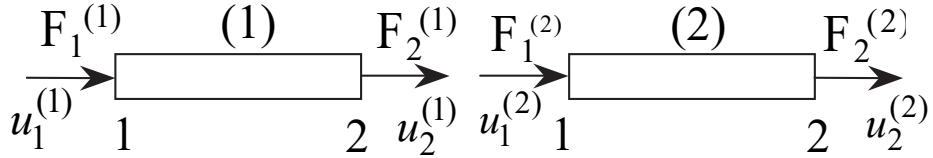
第2章
直接刚度法

2.3 系统总体刚度方程
Global Stiffness Equations

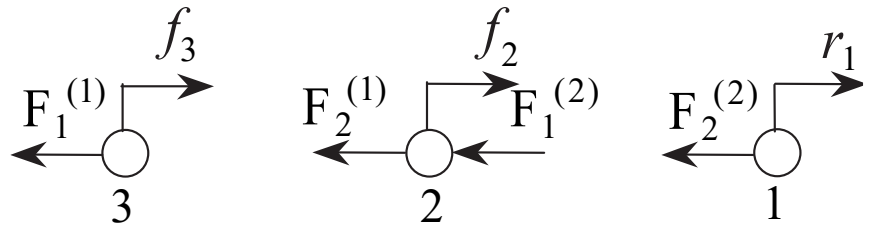
节点平衡方程







$$\boldsymbol{F}^{(1)} = \boldsymbol{K}^{(1)}\boldsymbol{d}^{(1)} \qquad \boldsymbol{F}^{(2)} = \boldsymbol{K}^{(2)}\boldsymbol{d}^{(2)}$$



将结构离散为若干个单元

- 载荷作用点处
- 截面或材料属性突变处
- 在每个节点处，给定外力或节点位移

刚度方程组装规则

- **节点位移协调条件：**所有单元在公共节点处的节点位移应该相等

$$u_1^{(1)} = u_3,\ u_2^{(1)} = u_2,\ u_1^{(2)} = u_2,\ u_2^{(2)} = u_1$$

- **节点力平衡条件：**所有单元作用在公共节点上的内力之和应该与作用在该节点上的外力平衡

$$F_2^{(2)} = r_1$$

$$F_2^{(1)} + F_1^{(2)} = f_2$$

$$F_1^{(1)} = f_3$$

节点平衡方程

各节点的平衡方程

$$F_2^{(2)} = r_1$$

$$F_2^{(1)} + F_1^{(2)} = f_2$$

$$F_1^{(1)} = f_3$$

$$\underbrace{\begin{bmatrix} 0 \\ F_2^{(1)} \\ F_1^{(1)} \end{bmatrix}}_{\tilde{\boldsymbol{F}}^{(1)}} + \underbrace{\begin{bmatrix} F_2^{(2)} \\ F_1^{(2)} \\ 0 \end{bmatrix}}_{\tilde{\boldsymbol{F}}^{(2)}} = \begin{bmatrix} r_1 \\ f_2 \\ f_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 \\ f_2 \\ f_3 \end{bmatrix}}_{\boldsymbol{f}} + \underbrace{\begin{bmatrix} r_1 \\ 0 \\ 0 \end{bmatrix}}_{\boldsymbol{r}}$$

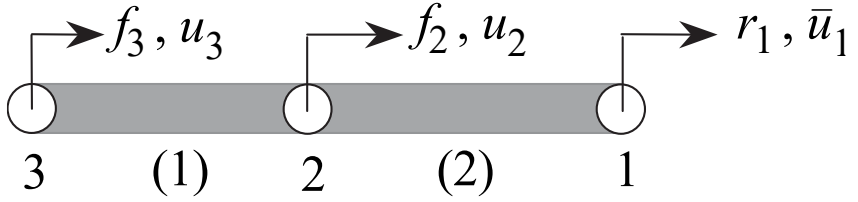
$$\tilde{\boldsymbol{F}}^{(1)} + \tilde{\boldsymbol{F}}^{(2)} = \boldsymbol{f} + \boldsymbol{r}$$

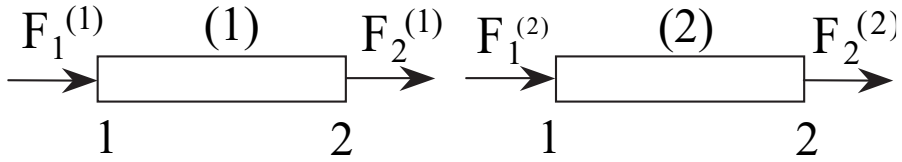
单元节点内力之和等于节点外力和约束反力之和

$\boldsymbol{F}^{(1)} \xrightarrow{?} \tilde{\boldsymbol{F}}^{(1)}$? → 矩阵散布　　1→3, 2→2

$\boldsymbol{F}^{(2)} \xrightarrow{?} \tilde{\boldsymbol{F}}^{(2)}$?　　1→2, 2→1

? → 单元刚度方程　　$\boldsymbol{F}^e = \boldsymbol{K}^e \boldsymbol{d}^e$





$$\boldsymbol{F}^{(1)} = \boldsymbol{K}^{(1)} \boldsymbol{d}^{(1)} \qquad \boldsymbol{F}^{(2)} = \boldsymbol{K}^{(2)} \boldsymbol{d}^{(2)}$$

$$\boldsymbol{F}^{(1)} = \begin{bmatrix} F_1^{(1)} \\ F_2^{(1)} \end{bmatrix} \qquad \boldsymbol{F}^{(2)} = \begin{bmatrix} F_1^{(2)} \\ F_2^{(2)} \end{bmatrix}$$

$$\boldsymbol{F}^e = \boldsymbol{K}^e \boldsymbol{d}^e$$

$$\begin{bmatrix} F_1^{(1)} \\ F_2^{(1)} \end{bmatrix} = \begin{bmatrix} k^{(1)} & -k^{(1)} \\ -k^{(1)} & k^{(1)} \end{bmatrix} \begin{bmatrix} u_3 \\ u_2 \end{bmatrix}$$

1→3, 2→2 散布

$$\underbrace{\begin{bmatrix} 0 \\ F_2^{(1)} \\ F_1^{(1)} \end{bmatrix}}_{\tilde{\boldsymbol{F}}^{(1)}} = \underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & k^{(1)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}}_{\tilde{\boldsymbol{K}}^{(1)}} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}_{\boldsymbol{d}}$$

$$\begin{bmatrix} F_1^{(2)} \\ F_2^{(2)} \end{bmatrix} = \begin{bmatrix} k^{(2)} & -k^{(2)} \\ -k^{(2)} & k^{(2)} \end{bmatrix} \begin{bmatrix} u_2 \\ u_1 \end{bmatrix}$$

1→2, 2→1 散布

$$\underbrace{\begin{bmatrix} F_2^{(2)} \\ F_1^{(2)} \\ 0 \end{bmatrix}}_{\tilde{\boldsymbol{F}}^{(2)}} = \underbrace{\begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(2)} & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{\tilde{\boldsymbol{K}}^{(2)}} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}_{\boldsymbol{d}}$$

$$\tilde{\boldsymbol{F}}^{(1)} + \tilde{\boldsymbol{F}}^{(2)} = \boldsymbol{f} + \boldsymbol{r} \Longrightarrow \boldsymbol{K}\boldsymbol{d} = \boldsymbol{f} + \boldsymbol{r}$$

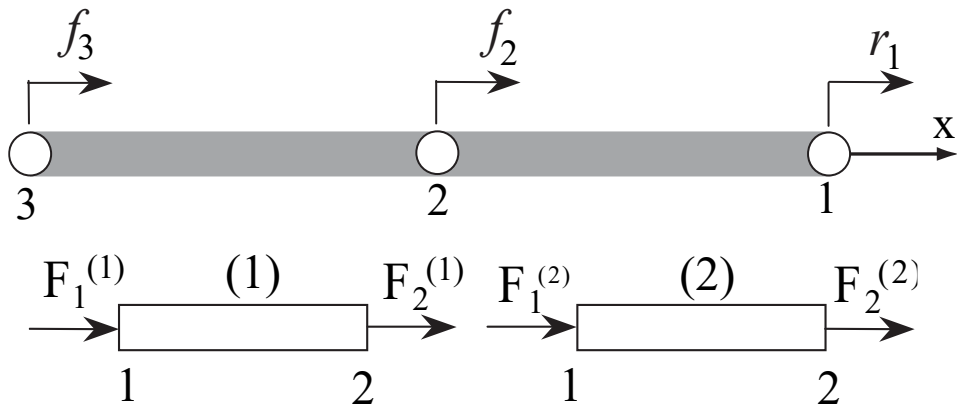
$$\boldsymbol{K} = \sum_{e=1}^{2} \tilde{\boldsymbol{K}}^{(e)}$$ 矩阵求和

$$= \begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)} + k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}$$

为什么？

为什么为零？

$\boldsymbol{K}$ — 总体刚度矩阵



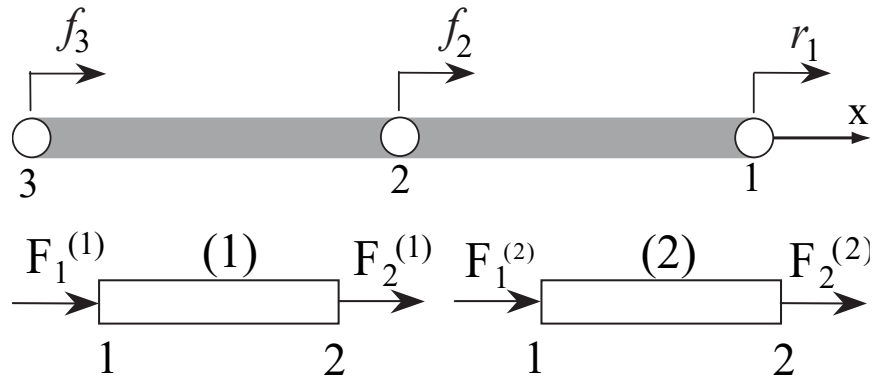
2.3 系统总体刚度方程

直接组装

可以根据单元自由度和总体自由度之间的对应关系，直接将单元刚度矩阵中的各元素累加到总体刚度矩阵的相应元素上，避免零元素求和运

$$\boldsymbol{K}^{(1)}=\begin{bmatrix} k^{(1)} & -k^{(1)} \\ -k^{(1)} & k^{(1)} \\ [3] & [2] \end{bmatrix}\begin{matrix}[3]\\[2]\end{matrix} \qquad \boldsymbol{K}^{(2)}=\begin{bmatrix} k^{(2)} & -k^{(2)} \\ -k^{(2)} & k^{(2)} \\ [2] & [1] \end{bmatrix}\begin{matrix}[2]\\[1]\end{matrix}$$

$$\boldsymbol{K}=\begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \\ [1] & [2] & [3] \end{bmatrix}\begin{matrix}[1]\\[2]\\[3]\end{matrix} \qquad \boldsymbol{K}=\mathop{\mathrm{A}}_{e=1}^{n_{\mathrm{el}}}\boldsymbol{K}^{e}$$



$$\mathrm{LM}=\begin{bmatrix} e{=}1 & e{=}2 \\ 3 & 2 \\ 2 & 1 \end{bmatrix}$$ — 对号矩阵

是否可改进？

```
import numpy as np
import FEData as model
def assembly(e, ke):
    for loop1 in range(model.nen*model.ndof):
        i = model.LM[loop1, e]
        for loop2 in range(model.nen*model.ndof):
            j = model.LM[loop2, e]
            model.K[i, j] += ke[loop1, loop2]
```

```
model.K[np.ix_(model.LM[:,e], model.LM[:,e])] += ke
```

Python代码: truss-python

https://github.com/xzhang66/FEM-Book

FEM-python/truss-python

Matlab 代码：truss-json

https://github.com/xzhang66/FEM-Book

FEM-MATLAB/truss-json

```
a[np.ix_([1,3],[2,5])] returns the array
[[a[1,2] a[1,5]],
 [a[3,2] a[3,5]]].
```

笛卡尔积（直积）

公式表达形式

$$\boldsymbol{d}^{(1)}=\begin{bmatrix}u_1^{(1)}\\u_2^{(1)}\end{bmatrix}=\underbrace{\begin{bmatrix}0&0&1\\0&1&0\end{bmatrix}}_{\boldsymbol{L}^{(1)}}\begin{bmatrix}\bar{u}_1\\u_2\\u_3\end{bmatrix}=\boldsymbol{L}^{(1)}\boldsymbol{d}$$

⇑ 提取矩阵

$$\boldsymbol{d}^{(2)}=\begin{bmatrix}u_1^{(2)}\\u_2^{(2)}\end{bmatrix}=\underbrace{\begin{bmatrix}0&1&0\\1&0&0\end{bmatrix}}_{\boldsymbol{L}^{(2)}}\begin{bmatrix}\bar{u}_1\\u_2\\u_3\end{bmatrix}=\boldsymbol{L}^{(2)}\boldsymbol{d}$$

$\boldsymbol{d}^e=\boldsymbol{L}^e\boldsymbol{d}$ —— 等价于施加位移协调条件

$$\boldsymbol{F}^e=\boldsymbol{K}^e\boldsymbol{d}^e=\boldsymbol{K}^e\boldsymbol{L}^e\boldsymbol{d}$$

$$\tilde{\boldsymbol{F}}^{(1)}=\begin{bmatrix}0\\F_2^{(1)}\\F_1^{(1)}\end{bmatrix}=\begin{bmatrix}0&0\\0&1\\1&0\end{bmatrix}\begin{bmatrix}F_1^{(1)}\\F_2^{(1)}\end{bmatrix}=\boldsymbol{L}^{(1)\mathrm{T}}\boldsymbol{F}^{(1)}$$

将节点力散布到总体矩阵中

$$\tilde{\boldsymbol{F}}^{(2)}=\begin{bmatrix}F_2^{(2)}\\F_1^{(2)}\\0\end{bmatrix}=\begin{bmatrix}0&1\\1&0\\0&0\end{bmatrix}\begin{bmatrix}F_1^{(2)}\\F_2^{(2)}\end{bmatrix}=\boldsymbol{L}^{(2)\mathrm{T}}\boldsymbol{F}^{(2)}$$

$$\tilde{\boldsymbol{F}}^e=\boldsymbol{L}^{e\mathrm{T}}\boldsymbol{F}^e$$

$$\tilde{\boldsymbol{F}}^{(1)}+\tilde{\boldsymbol{F}}^{(2)}=\boldsymbol{f}+\boldsymbol{r}$$

$$\sum_{e=1}^{n_{\mathrm{el}}}\boldsymbol{L}^{e\mathrm{T}}\boldsymbol{F}^e=\boldsymbol{f}+\boldsymbol{r}$$

$$\sum_{e=1}^{n_{\mathrm{el}}}\boldsymbol{L}^{e\mathrm{T}}\boldsymbol{K}^e\boldsymbol{L}^e\boldsymbol{d}=\boldsymbol{f}+\boldsymbol{r}$$

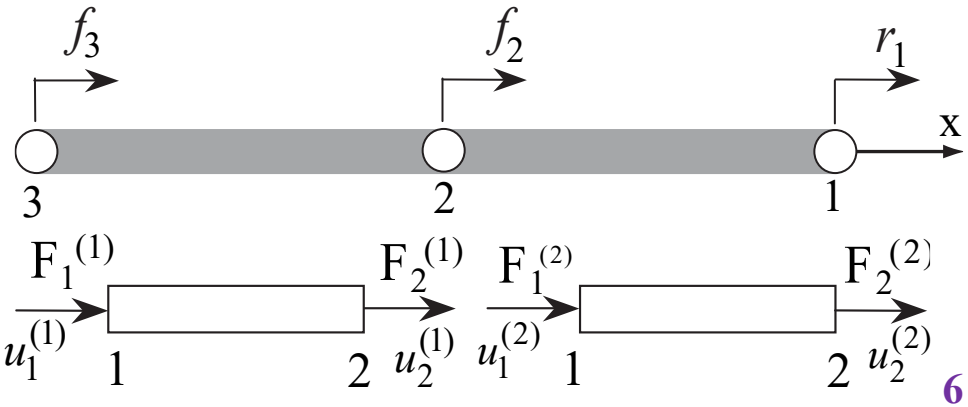
$$\boldsymbol{K}\boldsymbol{d}=\boldsymbol{f}+\boldsymbol{r}\qquad \boldsymbol{K}=\sum_{e=1}^{n_{\mathrm{el}}}\boldsymbol{L}^{e\mathrm{T}}\boldsymbol{K}^e\boldsymbol{L}^e$$

- 与矩阵散布求和等价
- 刚度矩阵散布对应于对 $\boldsymbol{K}^e$ 前乘 $\boldsymbol{L}^{e\mathrm{T}}$ 后乘 $\boldsymbol{L}^e$

$$\begin{bmatrix}k^{(2)}&-k^{(2)}&0\\-k^{(2)}&k^{(1)}+k^{(2)}&-k^{(1)}\\0&-k^{(1)}&k^{(1)}\end{bmatrix}\begin{bmatrix}\bar{u}_1\\u_2\\u_3\end{bmatrix}=\begin{bmatrix}r_1\\f_2\\f_3\end{bmatrix}$$

? 存在什么困难？

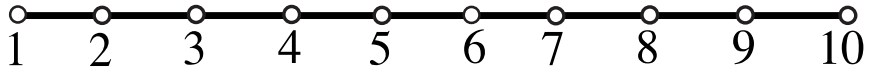
仅用于有限元格式的推导，实际编程时用直接组装法



总刚的物理意义和特性

- 总体刚度方程 $\boldsymbol{Kd}=\boldsymbol{f}+\boldsymbol{r}$ 给出了每个结点的平衡方程
- 总刚由单刚组装而成，因此也具有和单刚完全相同的物理意义和特性
  - 总体刚度矩阵第j列的物理意义？
    - ✓ 当结构的第j个自由度给定单位位移而其余自由度固定时，为了使结构平衡而需要在结构各自由度上施加的结点力
    - ✓ 二维隔行(列)元素之和为零；三维隔两行(列)元素之和为零
  - 总体刚度矩阵的特性？
    - ✓ 对称性
    - ✓ 奇异性
    - ✓ 对角元非负
    - ✓ **高度稀疏性**

下图所示的10结点一维系统的总体刚度矩阵有何特点？



10结点一维系统

边界条件

$$\begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}\begin{bmatrix} \bar{u}_1 \\ u_2 \\ u_3 \end{bmatrix}=\begin{bmatrix} r_1 \\ f_2 \\ f_3 \end{bmatrix}$$

- 总体刚度矩阵是奇异的
- 施加适当的位移边界条件后总体刚度矩阵非奇异
  - 缩减法
  - 修改法
  - 罚函数法

缩减法：将总体节点位移列阵分块为已知位移子列阵和未知位移子列阵两部分

$$\begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}\begin{bmatrix} \bar{u}_1 \\ u_2 \\ u_3 \end{bmatrix}=\begin{bmatrix} r_1 \\ f_2 \\ f_3 \end{bmatrix}$$

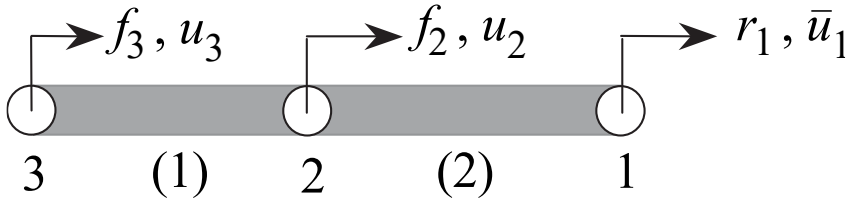
$$\begin{bmatrix} \boldsymbol{K}_{\mathrm{E}} & \boldsymbol{K}_{\mathrm{EF}} \\ \boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}} & \boldsymbol{K}_{\mathrm{F}} \end{bmatrix}\begin{bmatrix} \bar{\boldsymbol{d}}_{\mathrm{E}} \\ \boldsymbol{d}_{\mathrm{F}} \end{bmatrix}=\begin{bmatrix} \boldsymbol{r}_{\mathrm{E}} \\ \boldsymbol{f}_{\mathrm{F}} \end{bmatrix}$$

$$\boldsymbol{K}_{\mathrm{E}}\bar{\boldsymbol{d}}_{\mathrm{E}}+\boldsymbol{K}_{\mathrm{EF}}\boldsymbol{d}_{\mathrm{F}}=\boldsymbol{r}_{\mathrm{E}} \Longrightarrow \boldsymbol{r}_{\mathrm{E}}=\boldsymbol{K}_{\mathrm{E}}\bar{\boldsymbol{d}}_{\mathrm{E}}+\boldsymbol{K}_{\mathrm{EF}}\boldsymbol{d}_{\mathrm{F}}$$

$$\boldsymbol{K}_{\mathrm{F}}\boldsymbol{d}_{\mathrm{F}}=\boldsymbol{f}_{\mathrm{F}}-\boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}}\bar{\boldsymbol{d}}_{\mathrm{E}} \Longrightarrow \boldsymbol{d}_{\mathrm{F}}=\boldsymbol{K}_{\mathrm{F}}^{-1}(\boldsymbol{f}_{\mathrm{F}}-\boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}}\bar{\boldsymbol{d}}_{\mathrm{E}})$$

缩减刚度方程　　正定

$$-\boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}}\bar{\boldsymbol{d}}_{\mathrm{E}}=\begin{bmatrix} k^{(2)} \\ 0 \end{bmatrix}\bar{u}_1$$



- **修改法**：将与给定位移对应的刚度方程替换为相应的位移边界条件方程，并对其余方程进行相应的修正

$$\begin{bmatrix} k^{(2)} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}\begin{bmatrix} \bar{u}_1 \\ u_2 \\ u_3 \end{bmatrix}=\begin{bmatrix} r_1 \\ f_2 \\ f_3 \end{bmatrix} \Longrightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}\begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}=\begin{bmatrix} \bar{u}_1 \\ f_2-(-k^{(2)})\bar{u}_1 \\ f_3-(0)\bar{u}_1 \end{bmatrix}$$

修正刚度方程

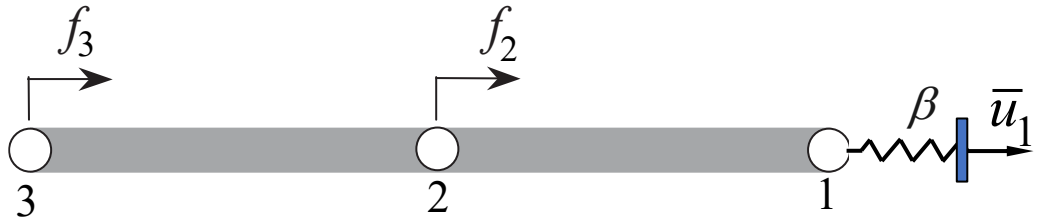
缩减刚度方程 $\boldsymbol{K}_{\mathrm{F}}\boldsymbol{d}_{\mathrm{F}}=\boldsymbol{f}_{\mathrm{F}}-\boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}}\bar{\boldsymbol{d}}_{\mathrm{E}}$

- **罚函数**：将与给定位移对应的刚度矩阵对角元替换为一个大数β，并将其右端项改为β与给定位移值的乘积

$$\begin{bmatrix} \boxed{\beta} & -k^{(2)} & 0 \\ -k^{(2)} & k^{(1)}+k^{(2)} & -k^{(1)} \\ 0 & -k^{(1)} & k^{(1)} \end{bmatrix}\begin{bmatrix} \boxed{u_1} \\ u_2 \\ u_3 \end{bmatrix}=\begin{bmatrix} \boxed{\beta\bar{u}_1} \\ f_2 \\ f_3 \end{bmatrix}$$

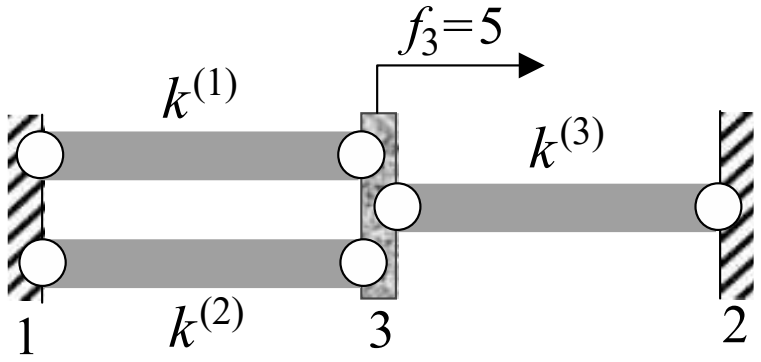
$$\beta \sim 10^7 \,\mathrm{average}(K_{ii})$$

对于应力分析问题，罚函数可以理解为在节点1和支座之间连接一个非常刚硬的弹簧，支座给定位移$\bar{u}_1$。



例题 2.1

如图所示，三杆铰接，其左端和右端均固定 (给定零位移)，并在中节点处作用5N的力。 节点编号从给定位移的节点开始。



各单元的刚度矩阵分别为

$$\boldsymbol{K}^{(1)}=\begin{bmatrix} k^{(1)} & -k^{(1)} \\ -k^{(1)} & k^{(1)} \end{bmatrix}\begin{matrix}[1]\\[3]\end{matrix} \qquad \boldsymbol{K}^{(2)}=\begin{bmatrix} k^{(2)} & -k^{(2)} \\ -k^{(2)} & k^{(2)} \end{bmatrix}\begin{matrix}[1]\\[3]\end{matrix}$$

$$\begin{matrix}[1] & [3]\end{matrix} \qquad \begin{matrix}[1] & [3]\end{matrix}$$

$$\boldsymbol{K}^{(3)}=\begin{bmatrix} k^{(3)} & -k^{(3)} \\ -k^{(3)} & k^{(3)} \end{bmatrix}\begin{matrix}[3]\\[2]\end{matrix}$$

$$\begin{matrix}[3] & [2]\end{matrix}$$

直接组装

$$\boldsymbol{K}=\begin{matrix}\begin{matrix}[1] & [2] & [3]\end{matrix}\\ \begin{bmatrix} k^{(1)}+k^{(2)} & 0 & -k^{(1)}-k^{(2)} \\ 0 & k^{(3)} & -k^{(3)} \\ -k^{(1)}-k^{(2)} & -k^{(3)} & k^{(1)}+k^{(2)}+k^{(3)} \end{bmatrix}\begin{matrix}[1]\\[2]\\[3]\end{matrix}\end{matrix}$$

$$\boldsymbol{d}=\begin{bmatrix} 0 \\ 0 \\ u_3 \end{bmatrix} \qquad \boldsymbol{f}=\begin{bmatrix} 0 \\ 0 \\ 5 \end{bmatrix} \qquad \boldsymbol{r}=\begin{bmatrix} r_1 \\ r_2 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} k^{(1)}+k^{(2)} & 0 & -k^{(1)}-k^{(2)} \\ 0 & k^{(3)} & -k^{(3)} \\ -k^{(1)}-k^{(2)} & -k^{(3)} & k^{(1)}+k^{(2)}+k^{(3)} \end{bmatrix}\begin{bmatrix} 0 \\ 0 \\ u_3 \end{bmatrix}=\begin{bmatrix} r_1 \\ r_2 \\ 5 \end{bmatrix}$$

$$\begin{bmatrix} \boldsymbol{K}_{\mathrm{E}} & \boldsymbol{K}_{\mathrm{EF}} \\ \boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}} & \boldsymbol{K}_{\mathrm{F}} \end{bmatrix}\begin{bmatrix} \bar{\boldsymbol{d}}_{\mathrm{E}} \\ \boldsymbol{d}_{\mathrm{F}} \end{bmatrix}=\begin{bmatrix} \boldsymbol{r}_{\mathrm{E}} \\ \boldsymbol{f}_{\mathrm{F}} \end{bmatrix}$$

$$(k^{(1)}+k^{(2)}+k^{(3)})u_3=5$$

$$u_3=\frac{5}{k^{(1)}+k^{(2)}+k^{(3)}}$$

$$\boldsymbol{K}_{\mathrm{E}}=\begin{bmatrix} k^{(1)}+k^{(2)} & 0 \\ 0 & k^{(3)} \end{bmatrix} \qquad \bar{\boldsymbol{d}}_{\mathrm{E}}=\begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\boldsymbol{K}_{\mathrm{EF}}=\begin{bmatrix} -k^{(1)}-k^{(2)} \\ -k^{(3)} \end{bmatrix} \qquad \boldsymbol{d}_{\mathrm{F}}=[u_3]$$

$$\boldsymbol{K}_{\mathrm{F}}=\left[k^{(1)}+k^{(2)}+k^{(3)}\right] \qquad \boldsymbol{r}_{\mathrm{E}}=\begin{bmatrix} r_1 \\ r_2 \end{bmatrix}$$

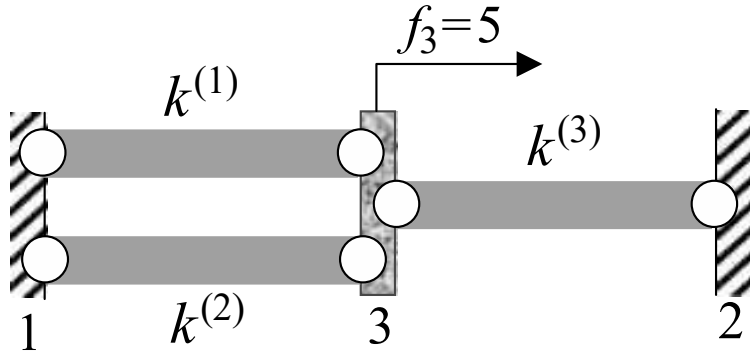
$$\boldsymbol{f}_{\mathrm{F}}=[5]$$

节点约束反力为

$$r_1=-\frac{5(k^{(1)}+k^{(2)})}{k^{(1)}+k^{(2)}+k^{(3)}}$$

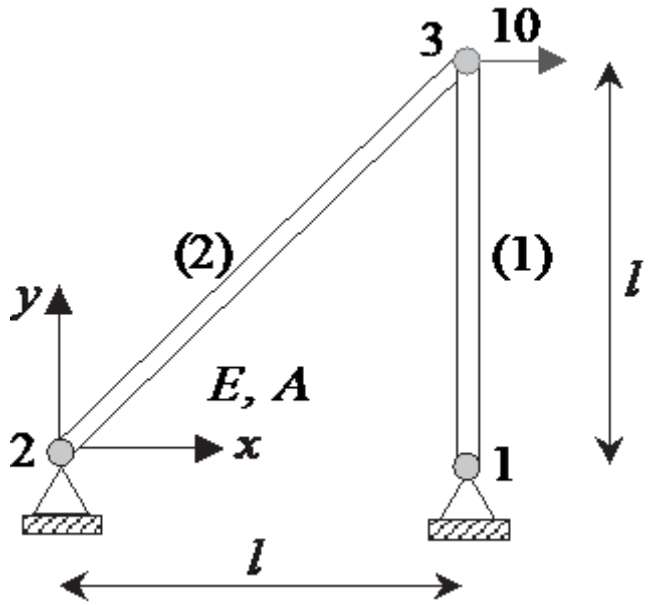
$$r_2=-\frac{5k^{(3)}}{k^{(1)}+k^{(2)}+k^{(3)}}$$

结果校核：$r_1+r_2+f_3=0$



例题 2.2

考虑如图所示的由两根杆组成的二维桁架结构，两杆的截面面积均为 A，弹性模量均为 E，几何尺寸、边界条件和载荷如图所示。求各结点的位移和各杆的应力。



例题 2.2

- 前处理
- 单元分析

$$\boldsymbol{K}^{(1)}=\frac{AE}{l}\begin{bmatrix}0&0&0&0\\0&1&0&-1\\0&0&0&0\\0&-1&0&1\end{bmatrix}\begin{matrix}[1]\\[2]\\[5]\\[6]\end{matrix}$$

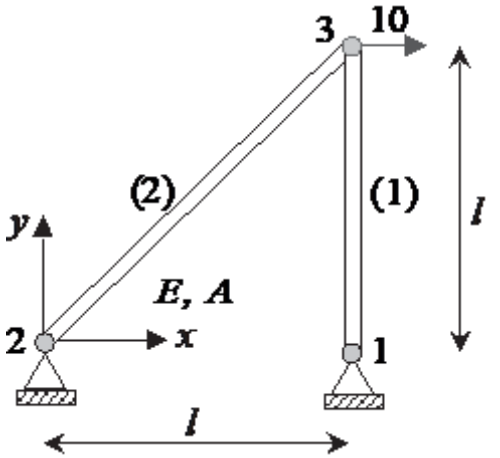
$$[1]\ [2]\ [5]\ [6]$$

$$\boldsymbol{K}^{(2)}=\frac{AE}{\sqrt{2}l}\begin{bmatrix}\frac{1}{2}&\frac{1}{2}&-\frac{1}{2}&-\frac{1}{2}\\\frac{1}{2}&\frac{1}{2}&-\frac{1}{2}&-\frac{1}{2}\\-\frac{1}{2}&-\frac{1}{2}&\frac{1}{2}&\frac{1}{2}\\-\frac{1}{2}&-\frac{1}{2}&\frac{1}{2}&\frac{1}{2}\end{bmatrix}\begin{matrix}[3]\\[4]\\[5]\\[6]\end{matrix}$$

$$[3]\ [4]\ [5]\ [6]$$

$$\mathrm{LM}=\begin{bmatrix}e=1&e=2\\1&3\\2&4\\5&5\\6&6\end{bmatrix}$$

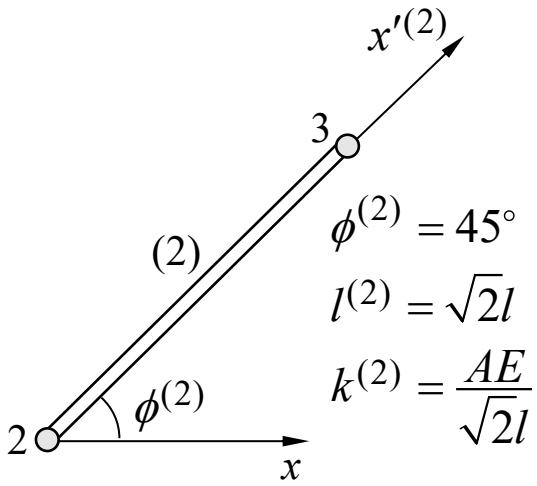
对号矩阵

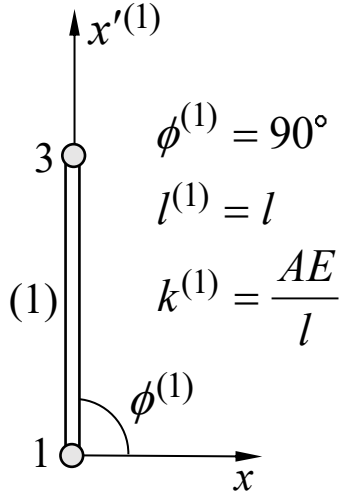


- 组装 $\boldsymbol{K}=?$

$$\boldsymbol{K}=\frac{AE}{l}\begin{bmatrix}0&0&0&0&0&0\\0&1&0&0&0&-1\\0&0&\frac{1}{2\sqrt{2}}&\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}\\0&0&\frac{1}{2\sqrt{2}}&\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}\\0&0&-\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}&\frac{1}{2\sqrt{2}}&\frac{1}{2\sqrt{2}}\\0&-1&-\frac{1}{2\sqrt{2}}&-\frac{1}{2\sqrt{2}}&\frac{1}{2\sqrt{2}}&1+\frac{1}{2\sqrt{2}}\end{bmatrix}\begin{matrix}[1]\\[2]\\[3]\\[4]\\[5]\\[6]\end{matrix}$$

$$[1]\ [2]\ [3]\ [4]\ [5]\ [6]$$





单元1: 1, 3

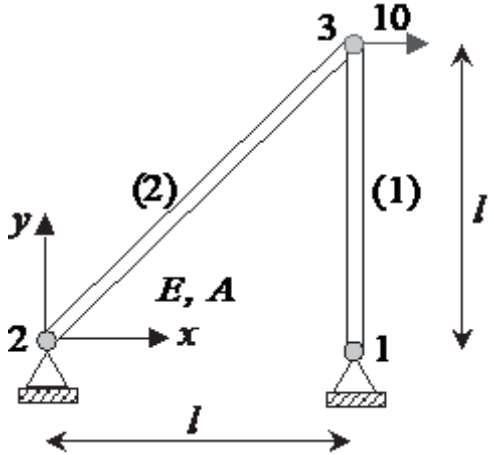
单元2: 2, 3

例题 2.2

- 施加位移边界条件，求解总刚度方程

$$\frac{AE}{l}\begin{bmatrix}
0 & 0 & 0 & 0 & 0 & 0\\
0 & 1 & 0 & 0 & 0 & -1\\
0 & 0 & \frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}}\\
0 & 0 & \frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}}\\
0 & 0 & -\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}}\\
0 & -1 & -\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}} & 1+\frac{1}{2\sqrt{2}}
\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\u_{3x}\\u_{3y}\end{bmatrix}=\begin{bmatrix}r_{1x}\\r_{1y}\\r_{2x}\\r_{2y}\\10\\0\end{bmatrix}$$

$$\begin{bmatrix}u_{3x}\\u_{3y}\end{bmatrix}=\frac{10l}{AE}\begin{bmatrix}1+2\sqrt{2}\\-1\end{bmatrix}$$



结果校核？

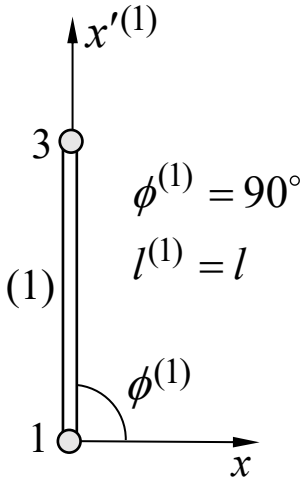
缩减系统刚度方程为

$$\frac{AE}{l}\begin{bmatrix}\frac{1}{2\sqrt{2}} & \frac{1}{2\sqrt{2}}\\ \frac{1}{2\sqrt{2}} & 1+\frac{1}{2\sqrt{2}}\end{bmatrix}\begin{bmatrix}u_{3x}\\u_{3y}\end{bmatrix}=\begin{bmatrix}10\\0\end{bmatrix}$$

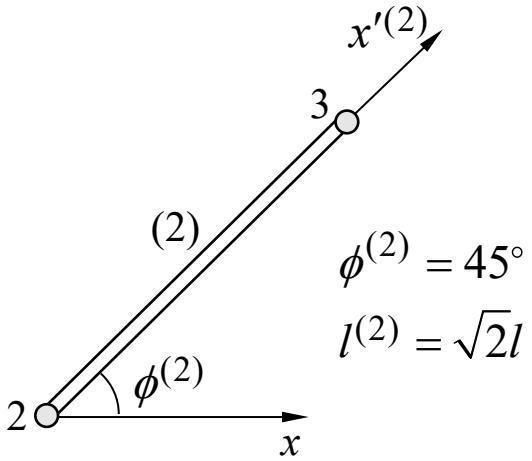
$$\begin{bmatrix}r_{1x}\\r_{1y}\\r_{2x}\\r_{2y}\end{bmatrix}=\frac{AE}{l}\begin{bmatrix}0 & 0\\0 & -1\\-\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}}\\-\frac{1}{2\sqrt{2}} & -\frac{1}{2\sqrt{2}}\end{bmatrix}\begin{bmatrix}u_{3x}\\u_{3y}\end{bmatrix}=\begin{bmatrix}0\\10\\-10\\-10\end{bmatrix}$$

- 后处理，计算单元应力

$$\sigma^e = E^e \boldsymbol{B}^e \boldsymbol{d}^e = \frac{E^e}{l^e}[-\cos\phi^e \ -\sin\phi^e \ \cos\phi^e \ \sin\phi^e]\boldsymbol{d}^e$$



$$\boldsymbol{d}^{(1)} = \begin{bmatrix} u_{1x} \\ u_{1y} \\ u_{3x} \\ u_{3y} \end{bmatrix} = \frac{l}{AE}\begin{bmatrix} 0 \\ 0 \\ 10+20\sqrt{2} \\ -10 \end{bmatrix} \qquad \sigma^{(1)} = -\frac{10}{A}$$



$$\boldsymbol{d}^{(2)} = \begin{bmatrix} u_{2x} \\ u_{2y} \\ u_{3x} \\ u_{3y} \end{bmatrix} = \frac{l}{AE}\begin{bmatrix} 0 \\ 0 \\ 10+20\sqrt{2} \\ -10 \end{bmatrix} \qquad \sigma^{(2)} = \frac{10\sqrt{2}}{A}$$

$$\begin{bmatrix} r_{1x} \\ r_{1y} \\ r_{2x} \\ r_{2y} \end{bmatrix} = \begin{bmatrix} 0 \\ 10 \\ -10 \\ -10 \end{bmatrix}$$

例题 2.2

Matlab 代码： truss-json

https://github.com/xzhang66/FEM-Book

FEM-MATLAB/truss-json

Python代码: truss-python

https://github.com/xzhang66/FEM-Book

FEM-python/truss-python

附录C.1 示例代码 truss-python

FEData.py: 定义有限元模型数据 (定义了22个变量)

Truss.py: 主程序

PrePost.py: 前处理模块 (读取有限元模型数据、生成对号数组、绘制结构图、计算单元应力等)

TrussElem.py: 一维和二维杆单元刚度矩阵

utitls.py: 单元刚度矩阵组装、方程求解

truss_2_1.json (本例)

truss_2_8.json

FEData.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import numpy as np

Title = ""
nsd   = 0
ndof  = 0
nnp   = 0
nel   = 0
nen   = 0
neq   = 0
nd    = 0

CArea = np.array([])
E     = np.array([])
leng  = np.array([])
stress= np.array([])

x     = np.array([])
y     = np.array([])
IEN   = np.array([[]])
LM    = np.array([[]])
K     = np.array([[]])
f     = np.array([[]])
d     = np.array([[]])

plot_truss = False
plot_node  = False
plot_tex   = False
```

例题 2.2

truss_2_1.json

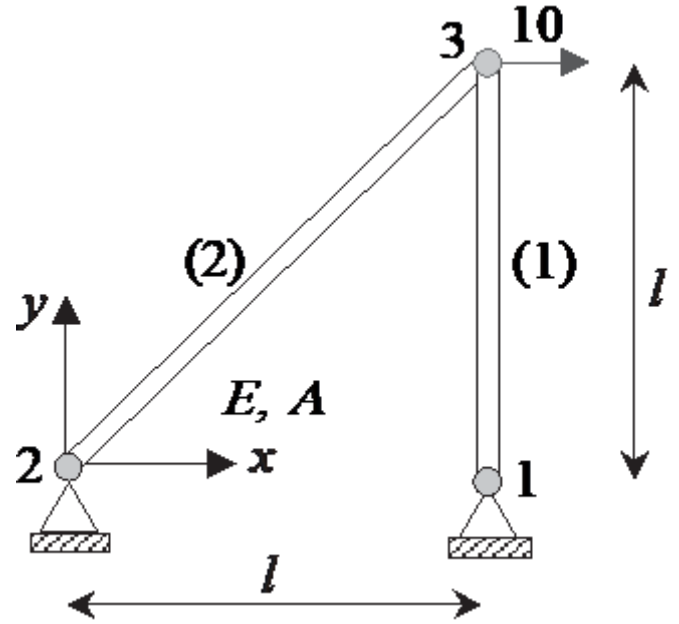
```
{
    "Title": " 2D truss Example",
    "nsd":  2,
    "ndof": 2,
    "nnp":  3,
    "nel":  2,
    "nen":  2,

    "CArea":  [1, 1],
    "E": [1, 1],

    "d": [0, 0, 0, 0],
    "nd": 4,

    "fdof": [5, 6],
    "force": [10.0, 0.0],

    "x": [1.0, 0.0, 1.0],
    "y": [0.0, 0.0, 1.0],

    "IEN": [
        [1, 3],
        [2, 3]
    ],

    "plot_truss": "yes",
    "plot_node" : "yes",
    "plot_tex"  : "yes"
}
```

JSON (JavaScript Object Notation, http://www.json.org) 是一种轻量级的数据交换格式。 易于人阅读和编写，也易于机器解析和生成。

- JSON 对象：以“{”开始以“}”结束的一个无序的‘键/值’对的集合
- 每个“键”后跟一个“:”，‘键/值’对之间用“,”分隔
- ‘值’可以是双引号括起来的字符串、数值、对象或者数组等，其中数组是值的有序集合，以“[”开始，以“]”结束，而值之间用“,”分隔
- 用import json导入json库后，利用FEData = json.load(open('test.json'))语句可将存储在文件'test.json'中的 JSON 对象载入到类型为 dict 的对象 FEData 中，然后利用FEData['键名']可获得该键的值

PrePost.py

```
import FEData as model
import json

def create_model_json(DataFile):

    with open(DataFile) as f_obj:
        FEData = json.load(f_obj)

    model.Title= FEData['Title']
    model.nsd  = FEData['nsd']
    model.ndof = FEData['ndof']
    model.nnp  = FEData['nnp']
    model.nel  = FEData['nel']
    model.nen  = FEData['nen']
    model.neq  = model.ndof*model.nnp
    model.nd   = FEData['nd']
 … …
```



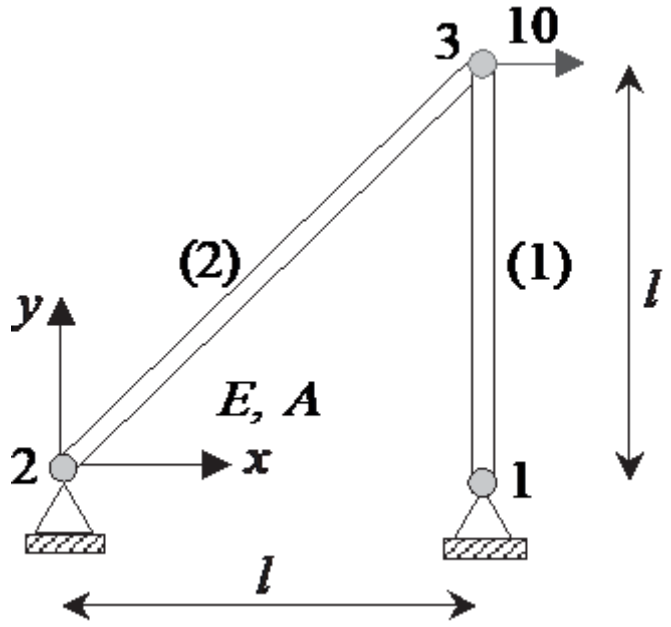
例题 2.2

Truss.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from sys import argv,exit
import FEData as model
from TrussElem import TrussElem
from PrePost import create_model_json, print_stress
from utitls import assembly, solvedr

def FERun(DataFile):
    # create FE model from DataFile in json format
    create_model_json(DataFile)  (1)

    # Element matrix computations and assembly
    for e in range(model.nel):
        ke = TrussElem(e)        (2)
        assembly(e, ke)          (3)

    # Partition and solution
    solvedr()                    (4)

    # Postprocessing
    print_stress()               (5)


if __name__ == "__main__":
    nargs = len(argv)
    if nargs == 2:
        DataFile = argv[1]
    else:
        print("Usage :  Truss file_name")
        exit()

    FERun(DataFile)
```

有限元法的5个主要步骤

1. 前处理
2. 单元分析 (单元刚度阵、单元节点力列阵)
3. 组装 (总体刚度阵、总体节点力列阵)
4. 方程求解
5. 后处理



命令行：**python Truss.py truss_2_1.json**

例题 2.2

(1) 前处理

$$x=\begin{bmatrix}1.0\\0.0\\1.0\end{bmatrix}\qquad y=\begin{bmatrix}0.0\\0.0\\1.0\end{bmatrix}\qquad IEN=\begin{bmatrix}e=1 & e=2\\1 & 2\\3 & 3\end{bmatrix}$$

单元节点号数组

总体节点号 = IEN(局部节点号, 单元号)

如何计算局部/全局自由度号？

$$LM[i,e]=2*(IEN[j,e]-1)+m$$

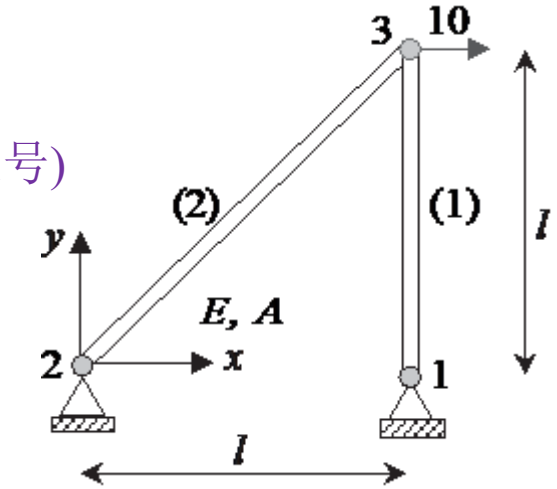
$$LM=\begin{bmatrix}e=1 & e=2\\1 & 3\\2 & 4\\5 & 5\\6 & 6\end{bmatrix}$$

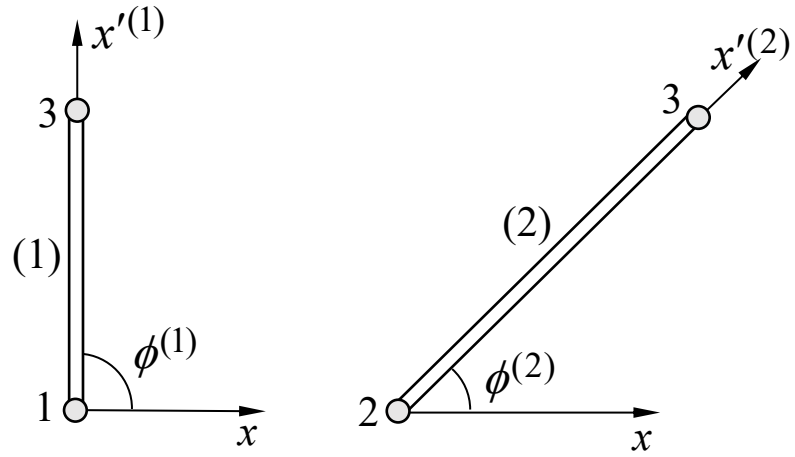
— 对号矩阵

$m=1,2$：节点自由度局部编号

$j=1,2$：节点局部编号

$i=2*(j-1)+m$：单元自由度全局编号





PrePost.py

```
def set_LM():
    '''
    set up Location Matrix
    '''
    for e in range(model.nel):
        for j in range(model.nen):
            for m in range(model.ndof):
                ind = j*model.ndof + m
                model.LM[ind, e] = model.ndof*(model.IEN[e, j] - 1) + m
```

- 在 Python 语言中，数组下标从 0 开始
- 函数 range(start,stop[, step]) 创建的整数列表不包含 stop，如 range(5) 创建的整数列表为[0,1,2,3,4]

(2) 计算单元矩阵

TrussElem.py

```
def TrussElem(e):
    # constant coefficient for each truss element
    const = model.CArea[e]*model.E[e]/model.leng[e]

    # calculate element stiffness matrix
    if model.ndof == 1:
        ke = const*np.array([[1, -1], [-1, 1]])
    elif model.ndof == 2:
        IENe = model.IEN[e] - 1
        xe = model.x[IENe]
        ye = model.y[IENe]
        s = (ye[1] - ye[0])/model.leng[e]
        c = (xe[1] - xe[0])/model.leng[e]

        s_s = s*s
        c_c = c*c
        c_s = c*s

        ke = const*np.array([[c_c, c_s, -c_c, -c_s],
                             [c_s, s_s, -c_s, -s_s],
                             [-c_c, -c_s, c_c, c_s],
                             [-c_s, -s_s, c_s, s_s]])
    elif model.ndof == 3:
        # insert your code here for 3D
        # ...
        pass # delete this line after your implementation for 3D
    else:
        raise ValueError("The dimension (ndof = {0}) given for the \
                          problem is invalid".format(model.ndof))

    return ke
```

$$\boldsymbol{K}^e = \frac{A^e E^e}{l^e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

增加3D Bar单元

(3) 直接组装

utitls.py

```
import numpy as np
import FEData as model


def assembly(e, ke):
    for loop1 in range(model.nen*model.ndof):
        i = model.LM[loop1, e]
        for loop2 in range(model.nen*model.ndof):
            j = model.LM[loop2, e]
            model.K[i, j] += ke[loop1, loop2]
```

是否可改进？

(4) 施加位移边界条件，求解总体刚度方程

utitls.py

```
def solvedr():
    """
    Partition and solve the system of equations

    Returns:
        f_E : (numpy.array(nd,1)) Reaction force vector
    """
    nd = model.nd; neq=model.neq
    K_E = model.K[0:nd, 0:nd]
    K_F = model.K[nd:neq, nd:neq]
    K_EF =model. K[0:nd, nd:neq]
    f_F = model.f[nd:neq]
    d_E = model.d[0:nd]

    # solve for d_F
    d_F = np.linalg.solve(K_F, f_F - K_EF.T @ d_E)

    # reconstruct the global displacement d
    model.d = np.append(d_E,d_F)

    # compute the reaction r
    f_E = K_E@d_E + K_EF@d_F

    # write to the workspace
    print('\nsolution d');  print(model.d)
    print('\nreaction f =', f_E)

    return f_E
```

@ — 矩阵乘法运算
python 3.5 or later

$$\begin{bmatrix} \boldsymbol{K}_{\mathrm{E}} & \boldsymbol{K}_{\mathrm{EF}} \\ \boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}} & \boldsymbol{K}_{\mathrm{F}} \end{bmatrix} \begin{bmatrix} \bar{\boldsymbol{d}}_{\mathrm{E}} \\ \boldsymbol{d}_{\mathrm{F}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{r}_{\mathrm{E}} \\ \boldsymbol{f}_{\mathrm{F}} \end{bmatrix}$$

$$\boldsymbol{d}_{\mathrm{F}} = \boldsymbol{K}_{\mathrm{F}}^{-1}(\boldsymbol{f}_{\mathrm{F}} - \boldsymbol{K}_{\mathrm{EF}}^{\mathrm{T}}\bar{\boldsymbol{d}}_{\mathrm{E}})$$

$$\boldsymbol{r}_{\mathrm{E}} = \boldsymbol{K}_{\mathrm{E}}\bar{\boldsymbol{d}}_{\mathrm{E}} + \boldsymbol{K}_{\mathrm{EF}}\boldsymbol{d}_{\mathrm{F}}$$

(5) 后处理

PrePost.py

```
def plottruss():
    '''
    plot the truss
    '''
    if model.plot_truss == "yes":
        if model.ndof == 1:
            for i in range(model.nel):
                XX = np.array([model.x[model.IEN[i, 0]-1],
                               model.x[model.IEN[i, 1]-1]])
                YY = np.array([0.0, 0.0])
                plt.plot(XX, YY, "blue")

                if model.plot_node == "yes":
                    plt.text(XX[0], YY[0], str(model.IEN[i, 0]))
                    plt.text(XX[1], YY[1], str(model.IEN[i, 1]))
… …
```

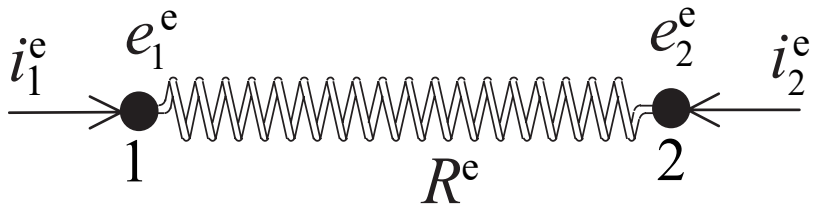
$$\boldsymbol{\sigma}^e = E^e \boldsymbol{B}^e \boldsymbol{d}^e$$

$$\boldsymbol{B}^e = \frac{1}{l^e}[-1 \ \ 1]\boldsymbol{T}^e$$

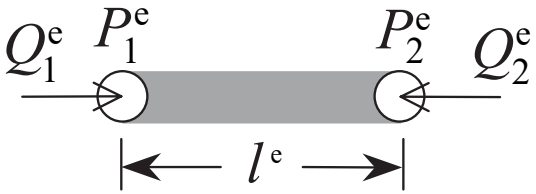
PrePost.py

```
def print_stress():

    # prints the element number and corresponding stresses
    print("Element\t\t\tStress")
    # Compute stress for each element
    for e in range(model.nel):
        de = model.d[model.LM[:, e]]  # nodal displacements
        const = model.E[e]/model.leng[e]

        if model.ndof == 1:
            model.stress[e] = const*(np.array([-1, 1])@de)
        elif model.ndof == 2:
            IENe = model.IEN[e] - 1
            xe = model.x[IENe]
            ye = model.y[IENe]
            s = (ye[1] - ye[0])/model.leng[e]
            c = (xe[1] - xe[0])/model.leng[e]
            model.stress[e] = const*(np.array([-c, -s, c, s])@de)
        elif model.ndof == 3:
            # insert your code here for 3D
            # ...
            pass # delete this line after your implementation for 3D
        else:
            raise ValueError("The dimension (ndof = {0}) given for the \
                              problem is invalid".format(model.ndof))

        print("{0}\t\t\t{1}".format(e+1, model.stress[e]))
```

- 具有以下特征的系统
  - 具有连续/协调的势
  - 通量和势(如电压、压力、温度和位移等)之间满足线性关系
  - 满足通量(如电流、流率、热流和内力等)守恒/平衡条件



- 电路中的稳态电流问题
  - 势 — 电压 e
  - 通量 — 电流 i
  - 欧姆定律 $i_2^e = (e_2^e - e_1^e)/R^e$
  - 电荷守恒 $i_1^e + i_2^e = 0$
- 管道系统中的流体流动问题
  - 势 — 压强 P
  - 通量 — 流率 Q
  - 流率-压强关系 $Q_2^e = \kappa^e (P_2^e - P_1^e)$
  - 流体守恒条件 $Q_1^e + Q_2^e = 0$



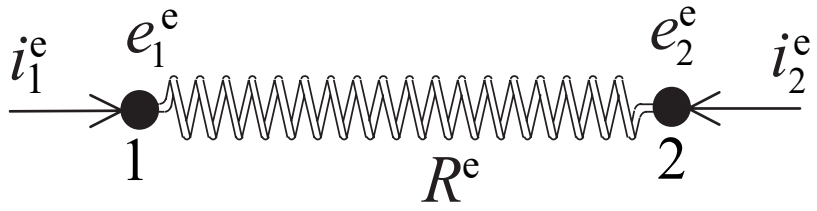
κ^e 取决于管道的截面面积、流体的粘性和单元的长度

欧姆定律

$$i_2^e = \frac{e_2^e - e_1^e}{R^e}$$

$$\underbrace{\begin{bmatrix} i_1^e \\ i_2^e \end{bmatrix}}_{\boldsymbol{F}^e} = \underbrace{\frac{1}{R^e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}}_{\boldsymbol{K}^e} \underbrace{\begin{bmatrix} e_1^e \\ e_2^e \end{bmatrix}}_{\boldsymbol{d}^e}$$

$$\boldsymbol{F}^e = \boldsymbol{K}^e \boldsymbol{d}^e$$



电荷守恒

$$i_1^e + i_2^e = 0$$

节点电压连续条件

$$\boldsymbol{d}^e = \boldsymbol{L}^e \boldsymbol{d}$$

节点电流平衡：节点处电流之和等于外部电流源

$$\sum_{e=1}^{n_{el}} \boldsymbol{L}^{e\mathrm{T}} \boldsymbol{F}^e = \boldsymbol{f} + \boldsymbol{r}$$

$$\boldsymbol{K}\boldsymbol{d} = \boldsymbol{f} + \boldsymbol{r} \qquad \boldsymbol{K} = \sum_{e=1}^{n_{el}} \boldsymbol{L}^{e\mathrm{T}} \boldsymbol{K}^e \boldsymbol{L}^e$$

不同线性系统之间的对应关系

	应力分析	电流分析	流体流动
势	位移	电压	压力
通量	内力	电流	流率

一维稳态热传导问题？

结束